Binary Tree Network

Dr.P.Ganesh Kumar
Anna University

Introduction

As parallel computing systems grow in size, the communication network must support efficient data exchange while maintaining low hardware complexity. One of the simplest hierarchical interconnection structures is the **Binary Tree Network**.

A binary tree organizes processors in a hierarchical parent–child relationship, where every processor (except the root) has one parent, and each internal processor has at most two children. This arrangement supports efficient communication for operations that naturally follow a tree structure, such as reduction, broadcasting, searching, and divide-and-conquer computations.

Binary tree networks are widely used in multiprocessor systems, distributed computing, parallel databases, and high-performance computing because they provide structured communication with relatively low hardware cost.

Definition

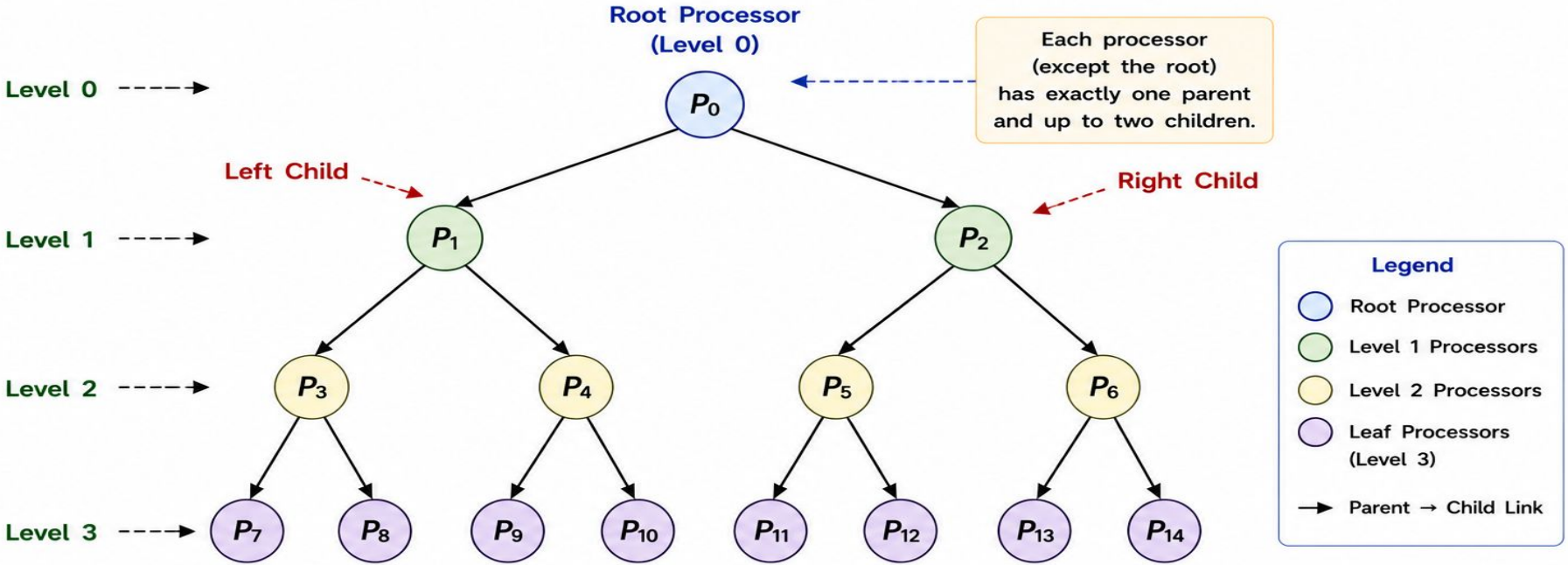
A **Binary Tree Network** is a **direct interconnection network** in which processing elements are organized as a binary tree. Each internal processor has one parent and up to two child processors, creating a hierarchical communication structure.

Characteristics

- Hierarchical topology
- One root node
- Every internal node has at most two children
- Leaf nodes have no children
- Supports recursive algorithms
- Low hardware complexity
- Efficient broadcast and reduction operations

Figure 3.1 – Basic Structure of a Binary Tree Network

Binary tree network organizes processors in a hierarchical parent–child relationship.



- The topmost processor is called the Root.
- Each processor at Level i has up to two children at Level $i+1$.
- The bottom level processors are called Leaf Nodes (they have no children).
- Communication occurs only along the parent–child links.

2. Need for Binary Tree Networks

Binary tree networks provide an efficient hierarchical communication structure that matches the needs of many parallel algorithms and large-scale systems.

Why Binary Tree Networks?

1. **Efficient Hierarchical Communication**
   Data can be communicated easily in two natural directions – from root to leaves (broadcast) and from leaves to root (reduction).

2. **Support for Divide-and-Conquer Algorithms**
   Many problems can be recursively divided into smaller subproblems and assigned to child processors.

   Example: Merge Sort, Binary Search, Matrix Multiplication, Fast Fourier Transform.

3. **Reduced Communication Overhead**
   Each processor communicates only with its parent and children.
   This reduces the number of long-distance links and simplifies routing.

4. **Scalable Design**
   The tree can be expanded by adding more levels. New processors are added as leaf nodes without affecting the existing structure.

5. **Simple and Deterministic Routing**
   Between any two processors, there is a unique path through their common ancestor. This avoids deadlocks and simplifies implementation.

6. **Efficient for Collective Operations**
   Operations like broadcast, reduction, prefix computation, and searching are very efficient in a tree structure.

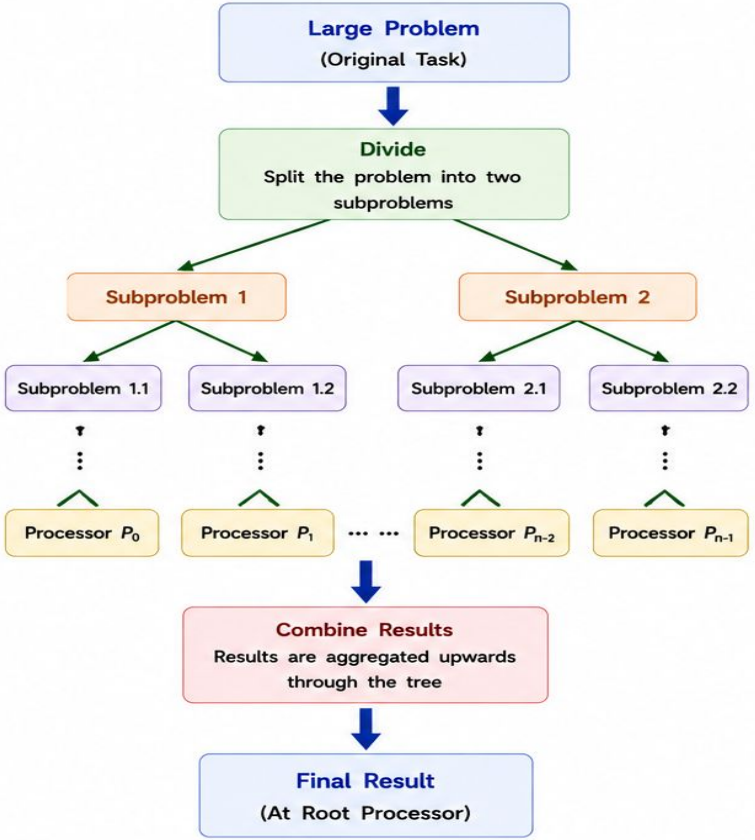
7. **Low Hardware Complexity**
   Binary tree networks require fewer links compared to fully connected or mesh structures, reducing cost and power consumption.

In Summary

Binary tree networks match the hierarchical nature of many applications and provide a good trade-off between performance, cost, and simplicity.

Figure 3.2 – Need for Binary Tree Network

Binary tree network enables hierarchical decomposition of a problem and efficient communication of results.



How it Works

- The large problem is at the root.
- It is divided into two smaller subproblems.
- Each subproblem is further divided recursively.
- Leaf processors solve the smallest subproblems.
- Results are combined upward until the final result is obtained at the root.

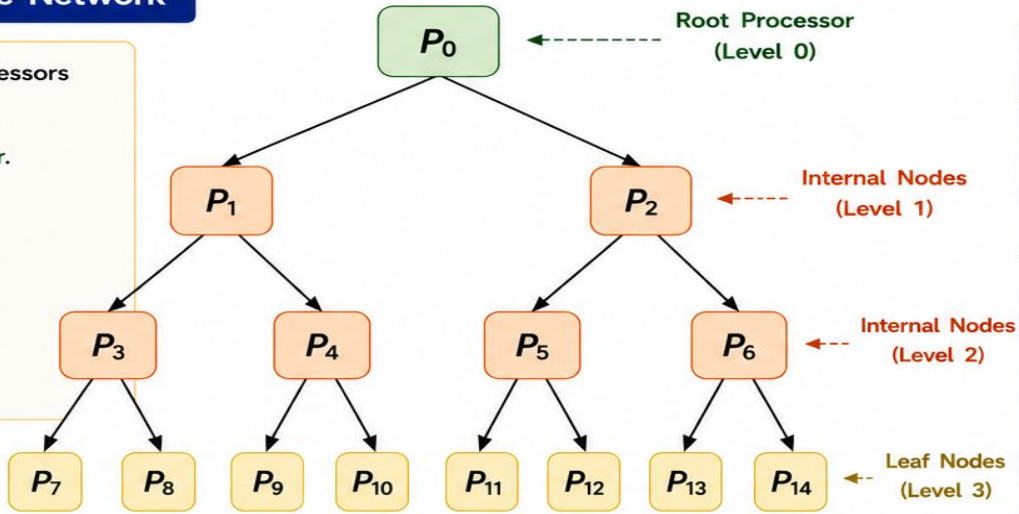
Key Benefit

Matches the recursive nature of many parallel applications.

3. Organization of Binary Tree Network

A Binary Tree Network organizes processors hierarchically.

- Level 0 contains the Root Processor.
- Each internal processor has two child processors.
- Every processor (except the root) has one parent.
- Leaf processors perform the final computation.



Levels in a Complete Binary Tree

Level (i)	Number of Nodes
Level 0	Root (1)
Level 1	2 Nodes
Level 2	4 Nodes
Level 3	8 Nodes
...	...

Complete Binary Tree Properties

- Number of Nodes (N): $N = 2^{h+1} - 1$
- Number of Leaf Nodes (L): $L = 2^h$
- Number of Internal Nodes (I): $I = 2^h - 1$

where · h = height of the tree (Root is at level 0)

- Height (h) is the number of edges on the longest path from the root to a leaf.
- For a complete binary tree, all levels are fully filled.

Example

- Height (h) = 3
- Number of Nodes (N) = $2^{3+1} - 1$ = **15**
- Leaf Nodes (L) = 2^3 = **8**
- Internal Nodes (I) = $2^3 - 1$ = **7**
- Levels = 0, 1, 2, 3 (Total **4** levels)

Figure 3.3 – Complete Binary Tree Organization

4. Components of Binary Tree Network

Each processor in a binary tree network consists of computational, memory and communication components that enable hierarchical data exchange.

1. **Processing Element (PE)**
   Executes the local computation assigned to the node. It may be a CPU core, microprocessor or any computational unit.

2. **Parent Link**
   A point-to-point communication link that connects the current processor to its parent processor (one link).

3. **Child Links**
   Point-to-point communication links that connect the current processor to its left and right child processors (up to two links).

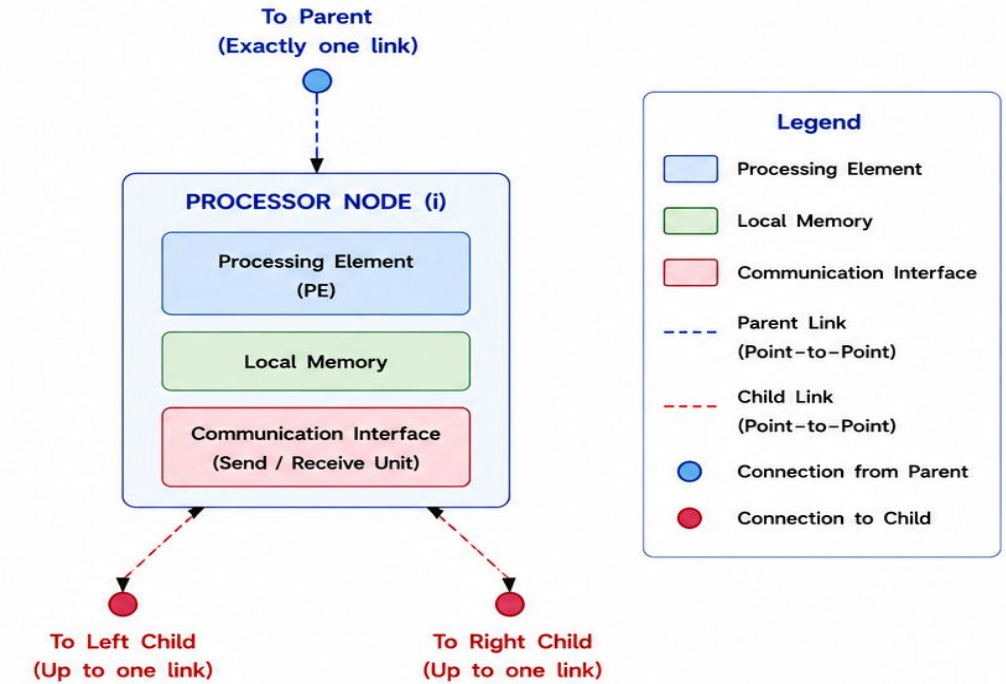
4. **Local Memory**
   Stores instructions, data and intermediate results required for local computation and communication.

5. **Communication Interface**
   Provides sending and receiving capability over parent and child links. It includes buffers, controllers and protocol handling units.

6. **Communication Channels**
   Physical or logical channels used to transfer data between connected processors (parent ↔ child).

Figure 3.4 – Components of Binary Tree Network

Each node (processor) communicates with its parent and up to two children through dedicated point-to-point links.



Key Points

- The root node has no parent link, only child links.
- Leaf nodes have no child links, only parent link.
- Internal nodes have one parent link and two child links.
- Communication is strictly along parent-child relationships.

5. Node Addressing Scheme

Each processor in a binary tree is assigned a unique index. The parent and child relationships are determined mathematically.

Addressing Rules (Array Representation)

- **Root Node** = 0
- **Left Child** = $2i + 1$
- **Right Child** = $2i + 2$
- **Parent** = $\left\lfloor \frac{(i - 1)}{2} \right\rfloor$

Example

Node (i) = 5

Parent = $\lfloor (5 - 1)/2 \rfloor$ = 2

Left Child = $2 \times 5 + 1$ = 11

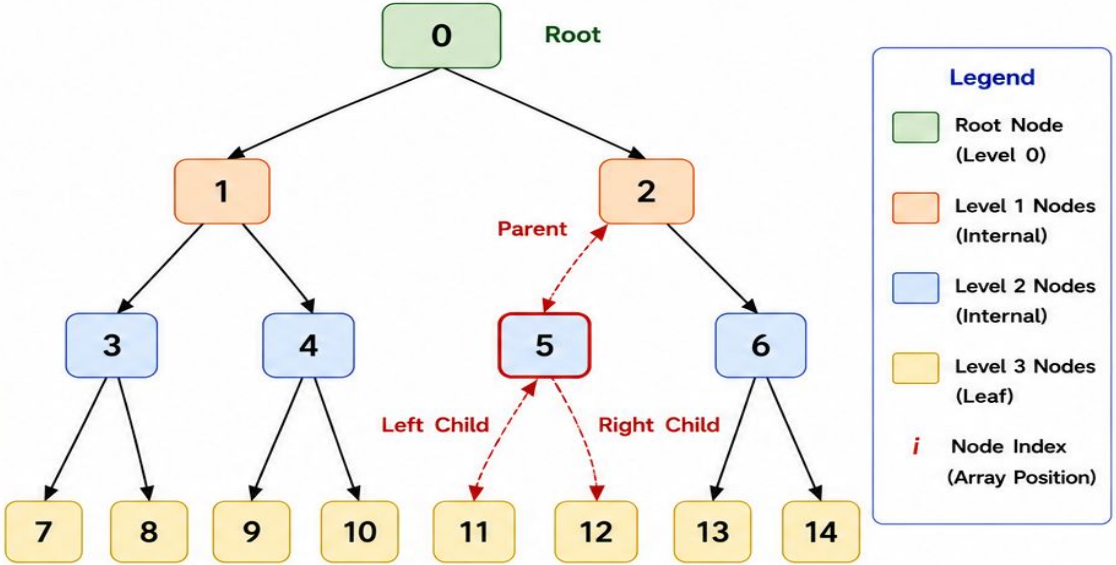
Right Child = $2 \times 5 + 2$ = 12

Note

This addressing scheme is widely used for implementing binary trees using arrays.

Figure 3.5 – Node Addressing in Binary Tree

Each node is labeled with its array index. Parent and child indices can be computed using the addressing rules.



Key Points

- Root node (0) has no parent.
- Every node (except root) has exactly one parent.
- Internal nodes have up to two children.
- Using array indices enables efficient navigation and easy implementation.

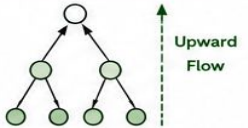
Figure 3.5 – Node Addressing in Binary Tree

6. Communication in Binary Tree Network

In a binary tree network, communication occurs only along the parent–child links. Data can flow in three basic forms.

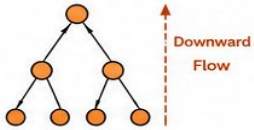
1 Upward Communication (Reduction)

- Data is sent from the leaf nodes up to the root.
- Used in operations such as summation, maximum, minimum, logical AND/OR, etc.
- Information gradually aggregates as it moves up.



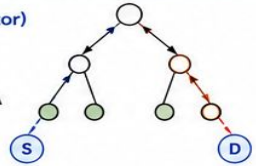
2 Downward Communication (Broadcast)

- Data is sent from the root down to all leaves.
- Used for distributing control information, common data, instructions, etc.
- Information is replicated as it moves down.



3 Peer Communication (Through Common Ancestor)

- Two nodes in different subtrees communicate via their lowest common ancestor (LCA).
- Data moves upward from the source to the LCA and then downward to the destination.
- Ensures unique path communication.



Important Properties

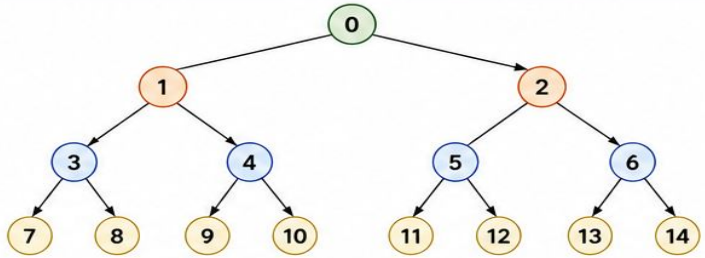
- Communication is always along parent–child links.
- There is exactly one unique path between any two nodes.
- Upward flow reduces data, downward flow distributes data.
- The root acts as the aggregation and distribution point.

Example Uses

- Broadcasting a task to all processors.
- Collecting partial results from processors.
- Parallel prefix operations.
- Tree-based algorithms (e.g., merge sort, tree search).

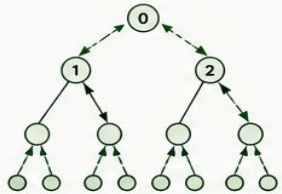
Figure 3.6 – Communication Paths

Communication between nodes in a binary tree network follows unique paths through parent–child links.



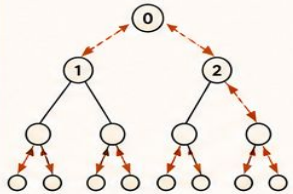
(a) Upward Communication (Reduction)

Data flows from leaves up to the root.



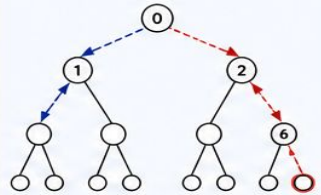
(b) Downward Communication (Broadcast)

Data flows from root down to all leaves.



(c) Peer Communication (Through LCA)

Communication between node 9 and node 14.



Example

To send data from node 11 to node 14:

Path: 11 → 5 → 2 → 6 → 14

Steps: 11 sends to 5 (up) → 5 sends to 2 (up) → 2 sends to 6 (down) → 6 sends to 14 (down).

Figure 3.6 – Communication Paths

7. Routing Algorithm

Routing in a binary tree network follows unique paths through parent–child links using the concept of Least Common Ancestor (LCA).

Routing Algorithm

Step 1: Starting from the source node, move upward through parent links until the Least Common Ancestor (LCA) of the source and destination is reached.

Step 2: From the LCA, move downward through child links until the destination node is reached.

Properties:

- There is exactly one unique path between any two nodes.
- Routing is deterministic and deadlock–free.
- The path length is at most twice the height of the tree ($2h$).

Finding Least Common Ancestor (LCA)

LCA of two nodes is the deepest node that is an ancestor of both the nodes.

Algorithm:

- Move upward from the deeper node until both nodes are at the same level.
- Move both nodes upward simultaneously until they meet.

Example (Illustrative)

Consider the binary tree in Figure 3.7.
Find the route from node 11 to node 14.

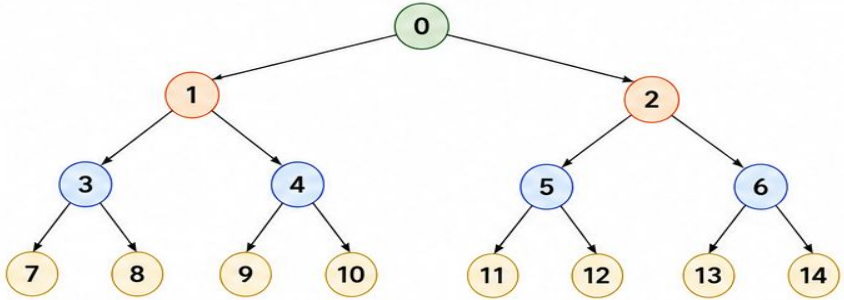
- Path from 11 to root: 11 → 5 → 2 → 0
- Path from 14 to root: 14 → 6 → 2 → 0
- LCA of 11 and 14 is node 2.
- Route: 11 → 5 → 2 → 6 → 14

Advantages of This Routing

- Simple to implement
- Requires only parent and child information
- Ensures minimal and unique path
- Well suited for broadcast, reduction and prefix operations

Figure 3.7 – Routing in Binary Tree

Routing between any two nodes is performed by first moving upward to the Least Common Ancestor (LCA) and then moving downward to the destination.



Example Route: From Node 11 to Node 14

Step 1: Move upward from source node 11 to LCA.

11 → 5 → 2 (LCA)
up up up

Step 2: Move downward from LCA (2) to destination 14.

2 → 6 → 14
down down down

Complete Route:

11 → 5 → 2 → 6 → 14
up up up down down

Another Example: From Node 7 to Node 10

Step 1: Move upward from 7 to LCA.

7 → 3 → 1 (LCA)
up up up

Step 2: Move downward from LCA (1) to 10.

1 → 4 → 10
down down down

Complete Route:

7 → 3 → 1 → 4 → 10
up up up down down

General Observations

- If both nodes are in the same subtree, the path goes up and then down within that subtree.
- If one node is ancestor of the other, the path is only upward or only downward.
- The maximum number of hops between any two nodes is $2h$, where h is the height of the tree.
- This routing strategy is widely used for collective communication operations.

Symbols Used

- → Direction of movement
- ↑ Upward (to parent)
- ↓ Downward (to child)

Figure 3.7 – Routing in Binary Tree

8. Performance Analysis

Performance analysis helps to evaluate the efficiency of a binary tree network in terms of its size, distance and connectivity.

Key Parameters and Formulas

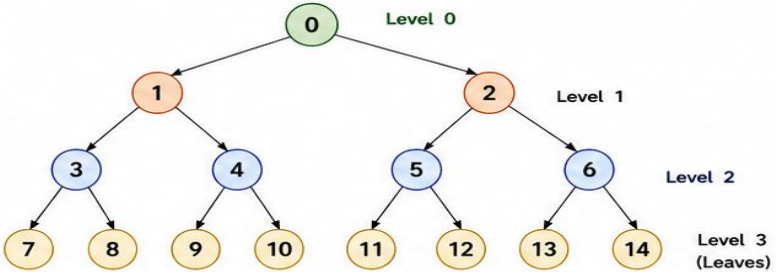
Parameter	Description	Formula (Complete Binary Tree)
Number of Nodes (N)	Total number of processors in the tree.	$N = 2^{h+1} - 1$
Number of Leaf Nodes (L)	Nodes at the lowest level (no children).	$L = 2^h$
Number of Internal Nodes (I)	Nodes that have at least one child.	$I = 2^h - 1$
Height (h)	Number of edges on the longest path from root to a leaf.	$h = \log_2(N + 1) - 1$
Diameter (D)	Maximum distance (in links) between any two nodes.	$D = 2h$
Degree of Nodes	Number of links (edges) connected to a node.	• Root node: 2 • Internal node: 3 • Leaf node: 1
Average Distance (AD)	Average number of links between two randomly chosen nodes.	$AD \approx O(\log N)$
Bisection Width (B_w)	Minimum number of links that must be removed to partition the network into two equal halves.	$B_w = 1$

Interpretation

- As the number of processors increases, the height grows logarithmically.
- Diameter is proportional to height, ensuring reasonably short communication paths.
- Degree is small (≤ 3), resulting in low hardware complexity.
- Bisection width is 1, which can become a communication bottleneck at large sizes.

Figure 3.8 – Performance Metrics of Binary Tree Network

Consider a complete binary tree of height $h = 3$ (15 nodes).



Computed Performance (for $h = 3$)

- Number of Nodes (N) = $2^{3+1} - 1 = 15$
- Leaf Nodes (L) = $2^3 = 8$
- Internal Nodes (I) = $2^3 - 1 = 7$
- Height (h) = 3
- Diameter (D) = $2h = 6$ } Longest path between any two nodes.
- Degree of Nodes = Root: 2, Internal: 3, Leaves: 1
- Average Distance (AD) ≈ $O(\log_2 15) \approx 4$
- Bisection Width (B_w) = 1

Distance Examples

- Between two leaves in different subtrees (e.g., 7 and 14):
  Path: 7 – 3 – 1 – 0 – 2 – 6 – 14
  Distance = 6 (= $2h$)
- Between nodes in same subtree (e.g., 9 and 10):
  Path: 9 – 4 – 10
  Distance = 2

Scalability Insight

For large N:

- Height $h \approx \log_2 N$
- Diameter $D \approx 2\log_2 N$
- Average distance grows logarithmically.

Strengths

- Low degree (≤ 3) → Simple hardware.
- Short paths (logarithmic) → Good scalability.
- Suitable for broadcast, reduction and divide-and-conquer operations.

Limitations

- Bisection width is only 1.
- Root becomes a bottleneck.
- Traffic near root increases with size.
- Not fault tolerant.

Figure 3.8 – Performance Metrics of Binary Tree Network

Advantages

- Simple hierarchical organization
- Efficient broadcast
- Efficient reduction
- Supports divide-and-conquer algorithms
- Low hardware complexity
- Easy implementation
- Deterministic routing
- Recursive structure

Disadvantages

- Root becomes a communication bottleneck
- Poor fault tolerance
- Long paths between distant leaves
- Limited scalability
- Small bisection bandwidth
- Heavy traffic near the root

Applications

Binary tree networks are widely used in:

Parallel Reduction
Summation
Maximum
Minimum

Broadcast Operations
Distributing common data

Parallel Sorting
Merge Sort
Tree Sort

Search Algorithms
Binary Search Trees
Decision Trees

Distributed Databases
Hierarchical indexing

Artificial Intelligence
Decision tree inference
Hierarchical clustering

Scientific Computing
Recursive numerical methods

Comparison with Other Topologies

Feature	Binary Tree	Mesh	Hypercube	Ring
Degree	≤3	2–4	$\log_2 N$	2
Diameter	$2\log_2 N$	Moderate	$\log_2 N$	N/2
Scalability	Good	Good	Excellent	Moderate
Cost	Low	Moderate	High	Low
Routing	Simple	Simple	Moderate	Simple

13. Worked Examples

Let us solve some typical problems based on binary tree networks.

Example 1: Node Addressing

Find the parent, left child and right child of node $i = 6$.

Using the addressing rules:

- Parent (6) = $\lfloor (6-1)/2 \rfloor = 2$
- Left Child (6) = $2 \times 6 + 1 = 13$
- Right Child (6) = $2 \times 6 + 2 = 14$

Example 2: Routing Between Two Nodes

Find the route from node 11 to node 14.

Step 1: Find LCA of 11 and 14.

Path from 11 to root: $11 \rightarrow 5 \rightarrow 2 \rightarrow 0$
Path from 14 to root: $14 \rightarrow 6 \rightarrow 2 \rightarrow 0$
Lowest Common Ancestor (LCA) = 2

Step 2: Route from 11 to 14 via LCA (2).

$11 \rightarrow 5 \rightarrow 2 \rightarrow 6 \rightarrow 14$

(Up, Up, Down, Down)

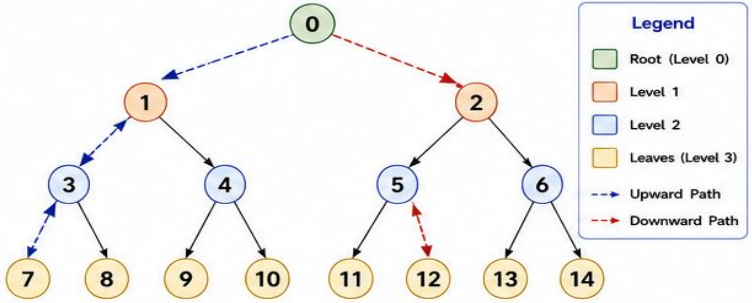
Example 3: Performance Metrics

For a binary tree of height $h = 4$.

- Number of Nodes: $N = 2^{4+1} - 1 = 31$
- Leaf Nodes: $L = 2^4 = 16$
- Internal Nodes: $I = 2^4 - 1 = 15$
- Height: $h = 4$
- Diameter: $D = 2h = 8$
- Maximum Hops between any two nodes: $2h = 8$

Figure 3.10 – Worked Example

Example: Find the route from node 7 to node 10 in the given binary tree.



Step	Upward to LCA (7 to LCA)			Step	Downward to Destination (LCA to 10)		
	From	To	Action		From	To	Action
1	7	3	Up (to parent)	4	0	2	Down (to child)
2	3	1	Up (to parent)	5	2	4	Down (to child)
3	1	0	Up (to parent)	6	4	10	Down (to child)

Complete Route

$7 \rightarrow 3 \rightarrow 1 \rightarrow 0 \rightarrow 2 \rightarrow 4 \rightarrow 10$

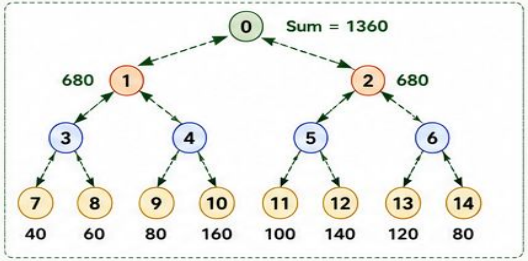
Total Hops = 6 ($\leq 2h$, where $h = 3$)

14. Real-World Case Study

Binary tree networks are used in many real systems for efficient and scalable communication.

Case Study: Parallel Reduction in HPC Systems

- In High Performance Computing (HPC), results from thousands of processors must be combined.
- A binary tree network is used for parallel reduction.
- Example: Summing 16 values using a binary tree.



How it works:

1. Leaf nodes hold individual values.
2. Each parent sums the values from its two children.
3. The root finally holds the total sum (1360).

Benefits:

- Scalable and efficient aggregation.
- Requires only short communication paths.
- Widely used in scientific simulations, machine learning (gradient aggregation), and distributed systems.

15. Summary

Key Takeaways

- A binary tree network organizes processors hierarchically.
- Each node (except root) has exactly one parent and up to two children.
- Communication occurs only along parent–child links.
- Routing between any two nodes uses the Least Common Ancestor (LCA).
- The network provides efficient support for broadcast, reduction and parallel operations.

Advantages

- Simple and regular structure.
- Logarithmic diameter → short communication paths.
- Low degree (≤ 3) → simple hardware.
- Good scalability and performance.
- Suitable for many parallel algorithms.

Limitations

- Bisection width is only 1.
- Root node can become a bottleneck.
- Performance depends on tree balance.
- Not ideal for all-to-all communication.

Applications

- Parallel reduction and prefix operations.
- Broadcast and collective communication.
- HPC systems and multiprocessors.
- Distributed databases and data aggregation.
- Machine learning and big data frameworks.